

LocalMind AI Deployment Setup Guide

Local development setup, environment configuration, production shape, and operational checks

This guide is based on the current repository. The project is configured for a local-first Django, React, Ollama, and ChromaDB workflow. Follow the local setup for development, then complete the production checklist before exposing the application publicly.

1 Prerequisites

- Python 3.11 or newer with a working virtual environment.
- Node.js 18 or newer and npm.
- Ollama installed and running on the local machine or a private reachable host.
- An Ollama model pulled before first use, for example `ollama pull phi3`.
- Optional scanned-PDF support: Tesseract OCR and Poppler.
- For production: PostgreSQL, HTTPS reverse proxy, durable file storage, and durable Chroma or another vector store.

2 Environment Variables

Variable	Local example	Purpose
SECRET_KEY	replace-with-random-secret	Django signing and security key
DEBUG	True locally; False in production	Debug mode switch
ALLOWED_HOSTS	127.0.0.1,localhost	Allowed host names
OLLAMA_BASE_URL	http://localhost:11434	Ollama API address
DEFAULT_AI_MODEL	phi3	Default local model
POPLER_PATH	optional Windows path	Poppler path for OCR conversion
TESSERACT_CMD	optional tesseract.exe path	Tesseract executable for OCR

Create a `.env` file at the repository root. Never commit `.env` or production secrets.

3 Windows Local Setup

```
cd E:\path\to\localmind-ai
python -m venv venv
.\venv\Scripts\Activate.ps1
python -m pip install -r requirements.txt
copy .env.example .env
python manage.py makemigrations
python manage.py migrate
python manage.py createsuperuser
python manage.py runserver
```

Open a second terminal for the frontend:

```
cd frontend
npm install
npm run dev
```

Open the Vite URL, normally `http://localhost:5173`. The Django API is normally available at `http://127.0.0.1:8000`.

Migration note

The repository currently ignores migration Python files. For team and production deployments, generate, review, and commit migrations so every environment applies the same schema history. Do not rely on a developer machine to recreate the schema during deployment.

4 Ollama Setup

Start Ollama, pull the configured model, and confirm the local API is available:

```
ollama serve
ollama pull phi3
ollama list
```

The application calls the Ollama `/api/generate` and `/api/tags` endpoints. The selected model must exist on the host configured by `OLLAMA_BASE_URL`.

5 First Run Smoke Test

- Open `/api/docs/` and confirm the OpenAPI page loads.
- Register a user, log in, refresh a token, and open the profile endpoint.
- Check the AI health endpoint and model listing.
- Create a session, send a prompt, refresh the page, and verify message persistence.
- Upload a small PDF, TXT, or DOCX; build the knowledge base; ask a grounded question; and confirm sources are returned.
- Create a prompt template, reuse it in chat, inspect dashboard counts, and test a TXT or JSON export.

6 Production Deployment Shape

The current codebase is appropriate for local development. A public deployment should use the following shape:

Component	Recommended shape	Required controls
Web API	Django ASGI served by Uvicorn behind Nginx or a managed proxy	HTTPS, health checks, timeouts, process restart
Frontend	npm run build served by CDN or Nginx	API base URL and immutable asset caching
Database	PostgreSQL	Backups, migrations, least-privilege account
AI	Private Ollama host with models pre-pulled	Capacity, timeouts, model health, access control
Files	Durable object or volume storage	Size limits, validation, backups, retention
Vectors	Durable Chroma volume or managed vector service	Backup, user filters, rebuild procedure
Jobs	Redis plus Celery for indexing and title generation	Retries, idempotency, status tracking

A production process should never depend on a laptop-local SQLite file, an ephemeral Chroma directory, or an Ollama endpoint exposed directly to the public internet.

7 Production Checklist

- Set `DEBUG=False`, a strong `SECRET_KEY`, exact `ALLOWED_HOSTS`, exact CORS origins, HTTPS, secure proxy headers, and secure cookies.
- Commit and apply migrations from a clean database before deployment.
- Replace SQLite for multi-user workloads and configure database backups.
- Persist media and vector data; test restore, deletion, and retention procedures.
- Enforce maximum upload size, allowed extensions, file validation, and private file access.
- Add tests, lint, frontend build checks, structured logs, request IDs, health probes, and error monitoring.
- Set Ollama resource limits and user/workspace rate limits; keep the AI host private.
- Review token storage, CORS, CSRF, security headers, admin access, ownership filters, and deletion behavior.

8 Troubleshooting

Symptom	Likely cause	Action
Django import error	Environment is inactive or dependencies are missing	Activate venv and install requirements
Ollama unavailable	Service stopped, URL wrong, or model missing	Start Ollama, check URL, run <code>ollama list/pull</code>
RAG has no context	Document is empty, unsupported, or not indexed	Check text extraction and build status
Scanned PDF has no text	OCR tools are absent or paths unset	Install Tesseract and Poppler and set paths
Frontend cannot call API	Backend, port, or CORS mismatch	Check API URL and CORS origins
Clean schema fails	Migrations are not committed	Generate and commit migrations

9 Launch Gate

LocalMind AI is ready for a portfolio demo or controlled beta after the core smoke test. Before public launch, complete clean migrations, production settings, automated tests, durable storage, upload security, monitoring, and robust Ollama failure handling. Complete workspace APIs, permissions, and UI before marketing the product as a collaboration platform.